

Tutte-Grothendieck Invariants for Graphs

María Alejandra Schild

Computational Complexity Seminar

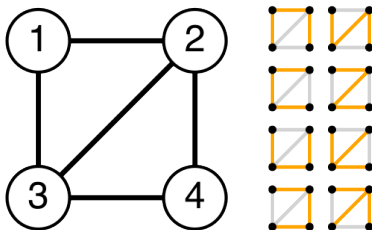
September 2025

Outline

- 1 Counting Problems and the Class $\#P$
- 2 Deletion and Contraction Operations
- 3 The Tutte Polynomial

A First Example: Spanning Trees

Let $G = (V, E)$ be an undirected, connected and simple graph (with no loops or parallel edges). Given $T \subseteq E$, the subgraph (V, T) is called a **spanning tree** of G if it is a tree that connects all vertices in V .



User: Andreschulz, Wikimedia Commons, CC BY-SA 3.0

Every spanning tree (V, T) satisfies $|T| = |V| - 1$.

Enumeration versus Counting

Enumeration problem: “Which ones?”

Is there a polynomial-time algorithm for **listing** the spanning trees of G ?

Enumeration versus Counting

Enumeration problem: “Which ones?”

Is there a polynomial-time algorithm for **listing** the spanning trees of G ?

No, because there may be too many.

Cayley's Formula

The complete graph K_n (with labeled vertices) has exactly n^{n-2} spanning trees.

Enumeration versus Counting

Enumeration problem: “Which ones?”

Is there a polynomial-time algorithm for **listing** the spanning trees of G ?

No, because there may be too many.

Cayley's Formula

The complete graph K_n (with labeled vertices) has exactly n^{n-2} spanning trees.

Counting problem: “How many?”

Is there a polynomial-time algorithm for **counting** the spanning trees of G ?

Enumeration versus Counting

Enumeration problem: “Which ones?”

Is there a polynomial-time algorithm for **listing** the spanning trees of G ?

No, because there may be too many.

Cayley's Formula

The complete graph K_n (with labeled vertices) has exactly n^{n-2} spanning trees.

Counting problem: “How many?”

Is there a polynomial-time algorithm for **counting** the spanning trees of G ?

Yes!

Kirchhoff's Matrix-Tree Theorem

Suppose $V = \{v_1, \dots, v_n\}$. The **Laplacian matrix** of $G = (V, E)$ is an $n \times n$ matrix defined as follows:

$$L_{ij} := \begin{cases} \deg(v_i) & \text{if } i = j \\ -1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise.} \end{cases}$$

Kirchhoff's Matrix-Tree Theorem

Suppose $V = \{v_1, \dots, v_n\}$. The **Laplacian matrix** of $G = (V, E)$ is an $n \times n$ matrix defined as follows:

$$L_{ij} := \begin{cases} \deg(v_i) & \text{if } i = j \\ -1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise.} \end{cases}$$

Kirchhoff's Theorem

Let L^* be the matrix obtained by deleting the first row and the first column of L . Then the number of spanning trees of G is $\det(L^*)$.

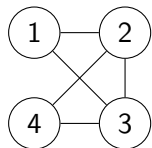
Kirchhoff's Matrix-Tree Theorem

Suppose $V = \{v_1, \dots, v_n\}$. The **Laplacian matrix** of $G = (V, E)$ is an $n \times n$ matrix defined as follows:

$$L_{ij} := \begin{cases} \deg(v_i) & \text{if } i = j \\ -1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise.} \end{cases}$$

Kirchhoff's Theorem

Let L^* be the matrix obtained by deleting the first row and the first column of L . Then the number of spanning trees of G is $\det(L^*)$.



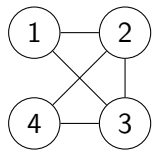
Kirchhoff's Matrix-Tree Theorem

Suppose $V = \{v_1, \dots, v_n\}$. The **Laplacian matrix** of $G = (V, E)$ is an $n \times n$ matrix defined as follows:

$$L_{ij} := \begin{cases} \deg(v_i) & \text{if } i = j \\ -1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise.} \end{cases}$$

Kirchhoff's Theorem

Let L^* be the matrix obtained by deleting the first row and the first column of L . Then the number of spanning trees of G is $\det(L^*)$.



$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 2 \end{pmatrix}$$

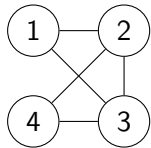
Kirchhoff's Matrix-Tree Theorem

Suppose $V = \{v_1, \dots, v_n\}$. The **Laplacian matrix** of $G = (V, E)$ is an $n \times n$ matrix defined as follows:

$$L_{ij} := \begin{cases} \deg(v_i) & \text{if } i = j \\ -1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise.} \end{cases}$$

Kirchhoff's Theorem

Let L^* be the matrix obtained by deleting the first row and the first column of L . Then the number of spanning trees of G is $\det(L^*)$.



$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 2 \end{pmatrix} \quad \begin{vmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 2 \end{vmatrix} = 8$$

A Second Example: Vertex Colorings

Let $G = (V, E)$ be a simple undirected graph, and let $k \geq 1$ be an integer. A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for all $u, v \in V$ with $\{u, v\} \in E$.

A Second Example: Vertex Colorings

Let $G = (V, E)$ be a simple undirected graph, and let $k \geq 1$ be an integer. A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for all $u, v \in V$ with $\{u, v\} \in E$.

Is there a polynomial-time algorithm for counting 1-colorings of G ?

A Second Example: Vertex Colorings

Let $G = (V, E)$ be a simple undirected graph, and let $k \geq 1$ be an integer. A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for all $u, v \in V$ with $\{u, v\} \in E$.

Is there a polynomial-time algorithm for counting 1-colorings of G ?

Yes. There are 0 if $E \neq \emptyset$, and 1 if $E = \emptyset$.

A Second Example: Vertex Colorings

Let $G = (V, E)$ be a simple undirected graph, and let $k \geq 1$ be an integer. A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for all $u, v \in V$ with $\{u, v\} \in E$.

Is there a polynomial-time algorithm for counting 1-colorings of G ?

Yes. There are 0 if $E \neq \emptyset$, and 1 if $E = \emptyset$.

Is there a polynomial-time algorithm for counting 2-colorings of G ?

A Second Example: Vertex Colorings

Let $G = (V, E)$ be a simple undirected graph, and let $k \geq 1$ be an integer. A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for all $u, v \in V$ with $\{u, v\} \in E$.

Is there a polynomial-time algorithm for counting 1-colorings of G ?

Yes. There are 0 if $E \neq \emptyset$, and 1 if $E = \emptyset$.

Is there a polynomial-time algorithm for counting 2-colorings of G ?

Yes. If G is not bipartite, it has no 2-colorings. If G is bipartite with $\kappa(G)$ connected components, then the number of 2-colorings of G is $2^{\kappa(G)}$.

A Second Example: Vertex Colorings

Let $G = (V, E)$ be a simple undirected graph, and let $k \geq 1$ be an integer. A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for all $u, v \in V$ with $\{u, v\} \in E$.

Is there a polynomial-time algorithm for counting 1-colorings of G ?

Yes. There are 0 if $E \neq \emptyset$, and 1 if $E = \emptyset$.

Is there a polynomial-time algorithm for counting 2-colorings of G ?

Yes. If G is not bipartite, it has no 2-colorings. If G is bipartite with $\kappa(G)$ connected components, then the number of 2-colorings of G is $2^{\kappa(G)}$.

Is there a polynomial-time algorithm for counting 3-colorings of G ?

A Second Example: Vertex Colorings

Let $G = (V, E)$ be a simple undirected graph, and let $k \geq 1$ be an integer. A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for all $u, v \in V$ with $\{u, v\} \in E$.

Is there a polynomial-time algorithm for counting 1-colorings of G ?

Yes. There are 0 if $E \neq \emptyset$, and 1 if $E = \emptyset$.

Is there a polynomial-time algorithm for counting 2-colorings of G ?

Yes. If G is not bipartite, it has no 2-colorings. If G is bipartite with $\kappa(G)$ connected components, then the number of 2-colorings of G is $2^{\kappa(G)}$.

Is there a polynomial-time algorithm for counting 3-colorings of G ?

Assuming $P \neq NP$, **no**, since even **deciding** whether a 3-coloring exists is NP-complete.

Polynomial-Time Counting Problems

A **counting problem** is a function $f: \{0, 1\}^* \rightarrow \mathbb{N}$.

Polynomial-Time Counting Problems

A **counting problem** is a function $f: \{0, 1\}^* \rightarrow \mathbb{N}$.

We say that a counting problem $f: \{0, 1\}^* \rightarrow \mathbb{N}$ is in the class **FP** if there is a deterministic Turing machine that runs in polynomial time and, on input $w \in \{0, 1\}^*$, computes the binary representation of $f(w)$.

Polynomial-Time Counting Problems

A **counting problem** is a function $f: \{0, 1\}^* \rightarrow \mathbb{N}$.

We say that a counting problem $f: \{0, 1\}^* \rightarrow \mathbb{N}$ is in the class **FP** if there is a deterministic Turing machine that runs in polynomial time and, on input $w \in \{0, 1\}^*$, computes the binary representation of $f(w)$.

Examples:

- Counting spanning trees is in FP.
- If $P \neq NP$, then counting 3-colorings is not in FP.

A Caveat About the Definition of FP

The class FP actually consists of all functions $f: \{0,1\}^* \rightarrow \{0,1\}^*$ computable in polynomial time. The output might naturally encode other kinds of objects.

Examples:

- Adding two rational numbers.
- Multiplying two matrices with integer entries.

A Caveat About the Definition of FP

The class FP actually consists of all functions $f: \{0,1\}^* \rightarrow \{0,1\}^*$ computable in polynomial time. The output might naturally encode other kinds of objects.

Examples:

- Adding two rational numbers.
- Multiplying two matrices with integer entries.

But we can transform any $f: \{0,1\}^* \rightarrow \{0,1\}^*$ into a counting problem $f': \{0,1\}^* \rightarrow \mathbb{N}$ so that, if $f(w) = a_k a_{k-1} \dots a_1 \in \{0,1\}^k$, then

$$f'(w) := 2^k + \sum_{i=1}^k 2^{i-1} a_i.$$

Notice that $f \in \text{FP}$ if and only if $f' \in \text{FP}$.

The Class #P

When we say that a nondeterministic Turing machine M runs in polynomial time, we assume that there is a polynomial $p(\cdot)$ such that, for every $w \in \{0, 1\}^*$, the number of steps of **any** computation of M on input w is at most $p(|w|)$.

The Class #P

When we say that a nondeterministic Turing machine M runs in polynomial time, we assume that there is a polynomial $p(\cdot)$ such that, for every $w \in \{0, 1\}^*$, the number of steps of **any** computation of M on input w is at most $p(|w|)$.

Given $w \in \{0, 1\}^*$, we define $\text{acc}_M(w)$ as the number of accepting computations of M on input w .

The Class #P

When we say that a nondeterministic Turing machine M runs in polynomial time, we assume that there is a polynomial $p(\cdot)$ such that, for every $w \in \{0, 1\}^*$, the number of steps of **any** computation of M on input w is at most $p(|w|)$.

Given $w \in \{0, 1\}^*$, we define $\text{acc}_M(w)$ as the number of accepting computations of M on input w .

Definition of the Class #P

A function $f : \{0, 1\}^* \rightarrow \mathbb{N}$ is in **#P** if and only if there is a nondeterministic Turing machine M that runs in polynomial time and such that, for every $w \in \{0, 1\}^*$, we have $f(w) = \text{acc}_M(w)$.

The Class #P

When we say that a nondeterministic Turing machine M runs in polynomial time, we assume that there is a polynomial $p(\cdot)$ such that, for every $w \in \{0,1\}^*$, the number of steps of **any** computation of M on input w is at most $p(|w|)$.

Given $w \in \{0,1\}^*$, we define $\text{acc}_M(w)$ as the number of accepting computations of M on input w .

Definition of the Class #P

A function $f : \{0,1\}^* \rightarrow \mathbb{N}$ is in **#P** if and only if there is a nondeterministic Turing machine M that runs in polynomial time and such that, for every $w \in \{0,1\}^*$, we have $f(w) = \text{acc}_M(w)$.

#3SAT, the problem of counting the truth assignments that satisfy a propositional formula in 3-CNF, is in #P.

#P Is Closed Under Addition and Multiplication

Let $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$ be functions in #P. Then:

- (1) $f + g \in \#P$.
- (2) $f \cdot g \in \#P$.

#P Is Closed Under Addition and Multiplication

Let $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$ be functions in #P. Then:

- (1) $f + g \in \#P$.
- (2) $f \cdot g \in \#P$.

Proof.

$f \in \#P$ via a nondeterministic Turing machine M_1 and $g \in \#P$ via another nondeterministic Turing machine M_2 .

#P Is Closed Under Addition and Multiplication

Let $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$ be functions in #P. Then:

- (1) $f + g \in \#P$.
- (2) $f \cdot g \in \#P$.

Proof.

$f \in \#P$ via a nondeterministic Turing machine M_1 and $g \in \#P$ via another nondeterministic Turing machine M_2 .

- For (1), consider a nondeterministic Turing machine that, on input $w \in \{0, 1\}^*$, guesses a bit $b \in \{0, 1\}$, simulates $M_{b+1}(w)$, and returns the answer of that simulation.

#P Is Closed Under Addition and Multiplication

Let $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$ be functions in #P. Then:

- (1) $f + g \in \#P$.
- (2) $f \cdot g \in \#P$.

Proof.

$f \in \#P$ via a nondeterministic Turing machine M_1 and $g \in \#P$ via another nondeterministic Turing machine M_2 .

- For (1), consider a nondeterministic Turing machine that, on input $w \in \{0, 1\}^*$, guesses a bit $b \in \{0, 1\}$, simulates $M_{b+1}(w)$, and returns the answer of that simulation.
- For (2), consider a nondeterministic Turing machine M_3 that, on input $w \in \{0, 1\}^*$, simulates $M_1(w)$. If the simulation rejects, then M_3 rejects; if the simulation accepts, then M_3 simulates $M_2(w)$ and returns the answer of that simulation.



FP versus #P

Recall that we can identify each element of $\mathbb{N}$ with its canonical binary representation. Conversely, we have already seen that every binary string canonically corresponds to a natural number.

FP versus #P

Recall that we can identify each element of $\mathbb{N}$ with its canonical binary representation. Conversely, we have already seen that every binary string canonically corresponds to a natural number.

$FP \subseteq \#P$, and the inclusion is strict if $P \neq NP$.

FP versus #P

Recall that we can identify each element of $\mathbb{N}$ with its canonical binary representation. Conversely, we have already seen that every binary string canonically corresponds to a natural number.

$\text{FP} \subseteq \#P$, and the inclusion is strict if $P \neq \text{NP}$.

Proof.

Let $f \in \text{FP}$. Notice that $|f(w)| = \text{poly}(|w|)$.

FP versus #P

Recall that we can identify each element of $\mathbb{N}$ with its canonical binary representation. Conversely, we have already seen that every binary string canonically corresponds to a natural number.

$\text{FP} \subseteq \#P$, and the inclusion is strict if $P \neq \text{NP}$.

Proof.

Let $f \in \text{FP}$. Notice that $|f(w)| = \text{poly}(|w|)$. Let M be a nondeterministic Turing machine that, on input $w \in \{0, 1\}^*$, computes $f(w)$, guesses a string $z \in \{0, 1\}^{|f(w)|}$, and accepts if and only if z is smaller than $f(w)$ in lexicographic order (with the most significant bit on the left).

FP versus #P

Recall that we can identify each element of $\mathbb{N}$ with its canonical binary representation. Conversely, we have already seen that every binary string canonically corresponds to a natural number.

$\text{FP} \subseteq \#P$, and the inclusion is strict if $P \neq \text{NP}$.

Proof.

Let $f \in \text{FP}$. Notice that $|f(w)| = \text{poly}(|w|)$. Let M be a nondeterministic Turing machine that, on input $w \in \{0, 1\}^*$, computes $f(w)$, guesses a string $z \in \{0, 1\}^{|f(w)|}$, and accepts if and only if z is smaller than $f(w)$ in lexicographic order (with the most significant bit on the left). Then $f \in \#P$ via M .

FP versus #P

Recall that we can identify each element of $\mathbb{N}$ with its canonical binary representation. Conversely, we have already seen that every binary string canonically corresponds to a natural number.

$FP \subseteq \#P$, and the inclusion is strict if $P \neq NP$.

Proof.

Let $f \in FP$. Notice that $|f(w)| = \text{poly}(|w|)$. Let M be a nondeterministic Turing machine that, on input $w \in \{0, 1\}^*$, computes $f(w)$, guesses a string $z \in \{0, 1\}^{|f(w)|}$, and accepts if and only if z is smaller than $f(w)$ in lexicographic order (with the most significant bit on the left). Then $f \in \#P$ via M .

For the second part, if $\#P = FP$, then $\#3SAT \in FP$, and we would have a polynomial-time algorithm for deciding 3SAT. □

FP versus #P

Recall that we can identify each element of $\mathbb{N}$ with its canonical binary representation. Conversely, we have already seen that every binary string canonically corresponds to a natural number.

$FP \subseteq \#P$, and the inclusion is strict if $P \neq NP$.

Proof.

Let $f \in FP$. Notice that $|f(w)| = \text{poly}(|w|)$. Let M be a nondeterministic Turing machine that, on input $w \in \{0, 1\}^*$, computes $f(w)$, guesses a string $z \in \{0, 1\}^{|f(w)|}$, and accepts if and only if z is smaller than $f(w)$ in lexicographic order (with the most significant bit on the left). Then $f \in \#P$ via M .

For the second part, if $\#P = FP$, then $\#3SAT \in FP$, and we would have a polynomial-time algorithm for deciding 3SAT. □

The equality $\#P = FP$ is regarded as more catastrophic than $NP = P$.

A First Notion of Reduction for Functions

Parsimonious Reductions

Given functions $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$, we say that f parsimoniously reduces to g (in polynomial time), denoted $f \leq_{par}^p g$, if there is an $h \in \text{FP}$ such that $f(w) = g(h(w))$ for every $w \in \{0, 1\}^*$.

A First Notion of Reduction for Functions

Parsimonious Reductions

Given functions $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$, we say that f parsimoniously reduces to g (in polynomial time), denoted $f \leq_{par}^p g$, if there is an $h \in \text{FP}$ such that $f(w) = g(h(w))$ for every $w \in \{0, 1\}^*$.

$\#3\text{SAT}$ is $\#P$ -complete under parsimonious reductions.

A First Notion of Reduction for Functions

Parsimonious Reductions

Given functions $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$, we say that f parsimoniously reduces to g (in polynomial time), denoted $f \leq_{par}^P g$, if there is an $h \in \text{FP}$ such that $f(w) = g(h(w))$ for every $w \in \{0, 1\}^*$.

$\#3\text{SAT}$ is $\#P$ -complete under parsimonious reductions.

The function $\#3\text{SAT} + 1$ is in $\#P$ because it is the sum of two functions in $\#P$, but it is **not** $\#P$ -complete under parsimonious reductions.

A First Notion of Reduction for Functions

Parsimonious Reductions

Given functions $f, g: \{0, 1\}^* \rightarrow \mathbb{N}$, we say that f parsimoniously reduces to g (in polynomial time), denoted $f \leq_{par}^P g$, if there is an $h \in \text{FP}$ such that $f(w) = g(h(w))$ for every $w \in \{0, 1\}^*$.

$\#3\text{SAT}$ is $\#P$ -complete under parsimonious reductions.

The function $\#3\text{SAT} + 1$ is in $\#P$ because it is the sum of two functions in $\#P$, but it is **not** $\#P$ -complete under parsimonious reductions.

But $\#3\text{SAT}$ and $\#3\text{SAT} + 1$ ought to be just as hard to compute!

Function Oracles

Machines with Oracles

Let $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a function. A Turing machine with an **oracle for f** has a special tape and a special state $q_?$. When it enters state $q_?$ and $x \in \{0, 1\}^*$ is the current content of the special tape, then, in one step, the content of that tape is replaced by $f(x) \in \{0, 1\}^*$, and the machine leaves state $q_?$.

Function Oracles

Machines with Oracles

Let $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a function. A Turing machine with an **oracle for f** has a special tape and a special state $q_?$. When it enters state $q_?$ and $x \in \{0, 1\}^*$ is the current content of the special tape, then, in one step, the content of that tape is replaced by $f(x) \in \{0, 1\}^*$, and the machine leaves state $q_?$.

Complexity Classes with Oracles

FP^f is the set of functions from $\{0, 1\}^*$ to $\{0, 1\}^*$ computable in polynomial time by a deterministic Turing machine with access to an oracle for f .

Function Oracles

Machines with Oracles

Let $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a function. A Turing machine with an **oracle for f** has a special tape and a special state $q_?$. When it enters state $q_?$ and $x \in \{0, 1\}^*$ is the current content of the special tape, then, in one step, the content of that tape is replaced by $f(x) \in \{0, 1\}^*$, and the machine leaves state $q_?$.

Complexity Classes with Oracles

FP^f is the set of functions from $\{0, 1\}^*$ to $\{0, 1\}^*$ computable in polynomial time by a deterministic Turing machine with access to an oracle for f .

Given a complexity class of functions $\mathcal{C}$, we define

$$\text{FP}^{\mathcal{C}} := \bigcup_{f \in \mathcal{C}} \text{FP}^f.$$

Function Oracles

Machines with Oracles

Let $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a function. A Turing machine with an **oracle for f** has a special tape and a special state $q_?$. When it enters state $q_?$ and $x \in \{0, 1\}^*$ is the current content of the special tape, then, in one step, the content of that tape is replaced by $f(x) \in \{0, 1\}^*$, and the machine leaves state $q_?$.

Complexity Classes with Oracles

FP^f is the set of functions from $\{0, 1\}^*$ to $\{0, 1\}^*$ computable in polynomial time by a deterministic Turing machine with access to an oracle for f .

Given a complexity class of functions $\mathcal{C}$, we define

$$\text{FP}^{\mathcal{C}} := \bigcup_{f \in \mathcal{C}} \text{FP}^f.$$

Exercise: If $f \in \text{FP}$, then $\text{FP}^f = \text{FP}$.

A More Flexible Notion of Reduction for Functions

Turing Reductions

Given $f, g: \{0,1\}^* \rightarrow \{0,1\}^*$, we say that f is Turing-reducible to g (in polynomial time), denoted $f \leq_T^p g$, if $f \in \text{FP}^g$.

A More Flexible Notion of Reduction for Functions

Turing Reductions

Given $f, g: \{0,1\}^* \rightarrow \{0,1\}^*$, we say that f is Turing-reducible to g (in polynomial time), denoted $f \leq_T^p g$, if $f \in \text{FP}^g$.

Every parsimonious reduction is a Turing reduction, but not conversely.

A More Flexible Notion of Reduction for Functions

Turing Reductions

Given $f, g: \{0,1\}^* \rightarrow \{0,1\}^*$, we say that f is Turing-reducible to g (in polynomial time), denoted $f \leq_T^P g$, if $f \in \text{FP}^g$.

Every parsimonious reduction is a Turing reduction, but not conversely.

Exercise

If $f_1 \leq_T^P f_2$ and $f_2 \leq_T^P f_3$, then $f_1 \leq_T^P f_3$.

A Better Notion of #P-hardness

#P-hardness and #P-completeness

- A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **#P-hard** (under Turing reductions) if $\#P \subseteq \text{FP}^f$.

A Better Notion of #P-hardness

#P-hardness and #P-completeness

- A function $f: \{0,1\}^* \rightarrow \{0,1\}^*$ is **#P-hard** (under Turing reductions) if $\#P \subseteq \text{FP}^f$.
- A counting problem $f: \{0,1\}^* \rightarrow \mathbb{N}$ is **#P-complete** if $f \in \#P$ and is #P-hard.

A Better Notion of #P-hardness

#P-hardness and #P-completeness

- A function $f: \{0,1\}^* \rightarrow \{0,1\}^*$ is **#P-hard** (under Turing reductions) if $\#P \subseteq \text{FP}^f$.
- A counting problem $f: \{0,1\}^* \rightarrow \mathbb{N}$ is **#P-complete** if $f \in \#P$ and is #P-hard.

If $\text{FP} \neq \#P$, then **no** #P-hard function can be computed in polynomial time.

A Better Notion of #P-hardness

#P-hardness and #P-completeness

- A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **#P-hard** (under Turing reductions) if $\#P \subseteq \text{FP}^f$.
- A counting problem $f: \{0, 1\}^* \rightarrow \mathbb{N}$ is **#P-complete** if $f \in \#P$ and is #P-hard.

If $\text{FP} \neq \#P$, then **no** #P-hard function can be computed in polynomial time.

#DNF is #P-complete.

It cannot happen that $\#3\text{SAT} \leq_{\text{par}}^p \#DNF$ unless $P = NP$.

A Better Notion of #P-hardness

#P-hardness and #P-completeness

- A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **#P-hard** (under Turing reductions) if $\#P \subseteq \text{FP}^f$.
- A counting problem $f: \{0, 1\}^* \rightarrow \mathbb{N}$ is **#P-complete** if $f \in \#P$ and is #P-hard.

If $\text{FP} \neq \#P$, then **no** #P-hard function can be computed in polynomial time.

#DNF is #P-complete.

It cannot happen that $\#3\text{SAT} \leq_{\text{par}}^P \# \text{DNF}$ unless $P = \text{NP}$.

But we do have $\#3\text{SAT} \leq_T^P \# \text{DNF}$: given φ in 3-CNF with n variables, we can write a DNF equivalent to $\neg\varphi$ and then use the fact that the number of truth assignments satisfying φ is $2^n - \# \text{DNF}(\neg\varphi)$.

The Power of $\#P$

$\#P$ -easiness and $\#P$ -equivalence

- A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is $\#P$ -easy if $f \in \text{FP}^{\#P}$.

The Power of #P

#P-easiness and #P-equivalence

- A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **#P-easy** if $f \in \text{FP}^{\#P}$.
- If f is both #P-easy and #P-hard, it is called **#P-equivalent**.

The Power of $\#P$

$\#P$ -easiness and $\#P$ -equivalence

- A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **$\#P$ -easy** if $f \in FP^{\#P}$.
- If f is both $\#P$ -easy and $\#P$ -hard, it is called **$\#P$ -equivalent**.

If $FP = \#P$, then **all** $\#P$ -easy functions can be computed in polynomial time.

The Power of $\#P$

$\#P$ -easiness and $\#P$ -equivalence

- A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **$\#P$ -easy** if $f \in \text{FP}^{\#P}$.
- If f is both $\#P$ -easy and $\#P$ -hard, it is called **$\#P$ -equivalent**.

If $\text{FP} = \#P$, then **all** $\#P$ -easy functions can be computed in polynomial time.

Examples: The following decision problems are $\#P$ -easy:

- Given a propositional formula, decide whether the number of truth assignments satisfying it is odd (a $\oplus P$ -complete problem).
- Given a propositional formula, decide whether more than half of all possible truth assignments satisfy it (a PP -complete problem).

The Power of $\#P$

$\#P$ -easiness and $\#P$ -equivalence

- A function $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **$\#P$ -easy** if $f \in FP^{\#P}$.
- If f is both $\#P$ -easy and $\#P$ -hard, it is called **$\#P$ -equivalent**.

If $FP = \#P$, then **all** $\#P$ -easy functions can be computed in polynomial time.

Examples: The following decision problems are $\#P$ -easy:

- Given a propositional formula, decide whether the number of truth assignments satisfying it is odd (a $\oplus P$ -complete problem).
- Given a propositional formula, decide whether more than half of all possible truth assignments satisfy it (a PP -complete problem).

Toda's Theorem

All decision problems in the polynomial hierarchy are $\#P$ -easy.

The Chromatic Function

From now on, $G = (V, E)$ will always be an undirected graph (not necessarily simple) with $|V| = n = n(G)$ and $|E| = m = m(G)$.

The Chromatic Function

From now on, $G = (V, E)$ will always be an undirected graph (not necessarily simple) with $|V| = n = n(G)$ and $|E| = m = m(G)$.

A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that for every edge with endpoints $u, v \in V$, we have $c(u) \neq c(v)$.

We denote the number of k -colorings of G by $\chi_G(k)$.

The Chromatic Function

From now on, $G = (V, E)$ will always be an undirected graph (not necessarily simple) with $|V| = n = n(G)$ and $|E| = m = m(G)$.

A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that for every edge with endpoints $u, v \in V$, we have $c(u) \neq c(v)$.

We denote the number of k -colorings of G by $\chi_G(k)$.

Evaluating $\chi_G(k)$ is in FP if $k \in \{1, 2\}$, and is #P-complete if $k \geq 3$.

The Chromatic Function

From now on, $G = (V, E)$ will always be an undirected graph (not necessarily simple) with $|V| = n = n(G)$ and $|E| = m = m(G)$.

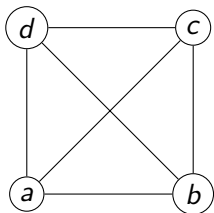
A **proper vertex k -coloring** of G is a function $c: V \rightarrow \{1, \dots, k\}$ such that for every edge with endpoints $u, v \in V$, we have $c(u) \neq c(v)$.

We denote the number of k -colorings of G by $\chi_G(k)$.

Evaluating $\chi_G(k)$ is in FP if $k \in \{1, 2\}$, and is #P-complete if $k \geq 3$.

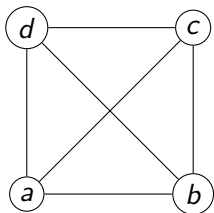
$\chi_G(k) \leq k^n$, with equality if and only if $m = 0$.

Chromatic Function of a Complete Graph



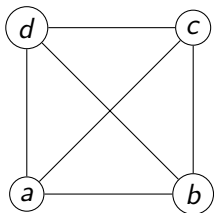
k	$\chi_G(k)$
-----	-------------

Chromatic Function of a Complete Graph



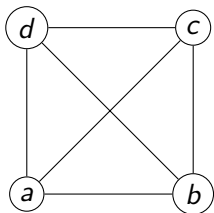
k	$\chi_G(k)$
1	0

Chromatic Function of a Complete Graph



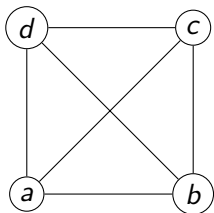
k	$\chi_G(k)$
1	0
2	0

Chromatic Function of a Complete Graph



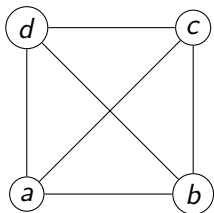
k	$\chi_G(k)$
1	0
2	0
3	0

Chromatic Function of a Complete Graph



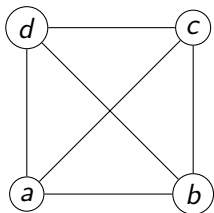
k	$\chi_G(k)$
1	0
2	0
3	0
4	$4 \cdot 3 \cdot 2 \cdot 1$

Chromatic Function of a Complete Graph



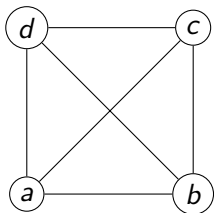
k	$\chi_G(k)$
1	0
2	0
3	0
4	$4 \cdot 3 \cdot 2 \cdot 1$
5	$5 \cdot 4 \cdot 3 \cdot 2$

Chromatic Function of a Complete Graph



k	$\chi_G(k)$
1	0
2	0
3	0
4	$4 \cdot 3 \cdot 2 \cdot 1$
5	$5 \cdot 4 \cdot 3 \cdot 2$
6	$6 \cdot 5 \cdot 4 \cdot 3$

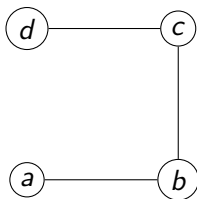
Chromatic Function of a Complete Graph



k	$\chi_G(k)$
1	0
2	0
3	0
4	$4 \cdot 3 \cdot 2 \cdot 1$
5	$5 \cdot 4 \cdot 3 \cdot 2$
6	$6 \cdot 5 \cdot 4 \cdot 3$

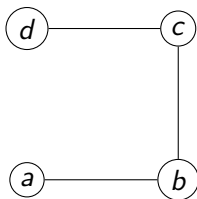
$$\chi_G(k) = k(k-1)(k-2)(k-3).$$

More Examples of Chromatic Functions

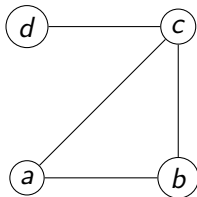


$$\chi_G(k) = k(k-1)^3$$

More Examples of Chromatic Functions

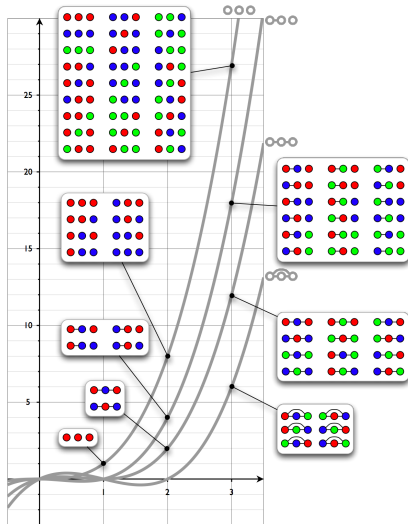


$$\chi_G(k) = k(k-1)^3$$



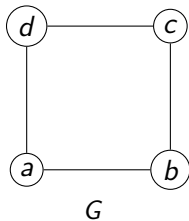
$$\chi_G(k) = k(k-1)^2(k-2)$$

Are Chromatic Functions Always Polynomials?

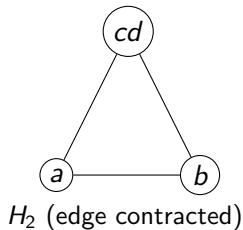
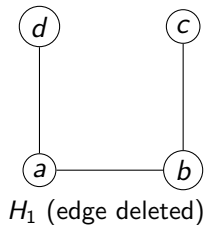
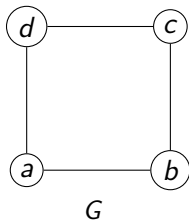


User: Thore Husfeldt, Wikimedia Commons, CC BY-SA 4.0

A More Complicated Example

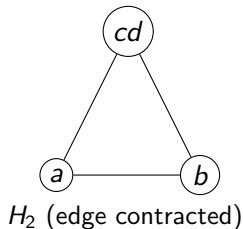
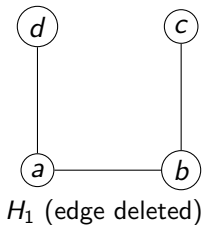
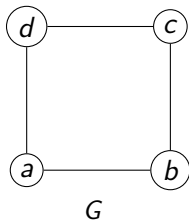


A More Complicated Example



$$\chi_{H_1}(k) = \chi_G(k) + \chi_{H_2}(k)$$

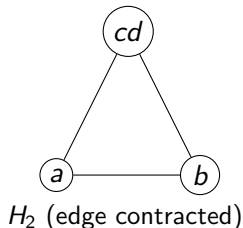
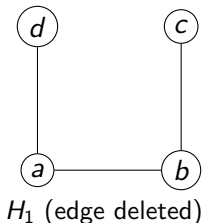
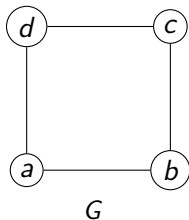
A More Complicated Example



$$\chi_{H_1}(k) = \chi_G(k) + \chi_{H_2}(k)$$

$$\chi_G(k) = \chi_{H_1}(k) - \chi_{H_2}(k)$$

A More Complicated Example



$$\chi_{H_1}(k) = \chi_G(k) + \chi_{H_2}(k)$$

$$\chi_G(k) = \chi_{H_1}(k) - \chi_{H_2}(k)$$

Since $m(H_1) = m(H_2) = m(G) - 1$, we have a **recurrence**.

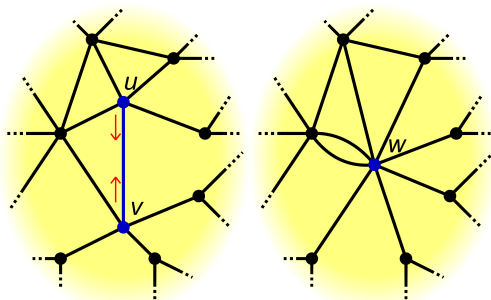
The Deletion and Contraction Operations

Given an edge $e \in E$, the **deletion** $G \setminus e$ is the graph $(V, E \setminus \{e\})$.

The Deletion and Contraction Operations

Given an edge $e \in E$, the **deletion** $G \setminus e$ is the graph $(V, E \setminus \{e\})$.

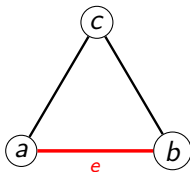
Given an edge $e \in E$ connecting distinct vertices u and v , the **contraction** G/e is the graph obtained by deleting the edge e and then identifying u and v as a single new vertex w .



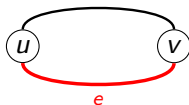
User: Trevorgunn, Wikimedia Commons, CC BY-SA 4.0

Watch Out for Loops

Contracting an edge of a triangle creates **parallel edges**

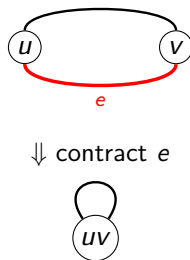
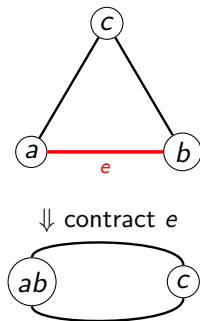


Contracting one of several parallel edges creates **loops**



Watch Out for Loops

Contracting an edge of a triangle creates **parallel edges**
Contracting one of several parallel edges creates **loops**



If G is a graph with a loop, then $\chi_G(k) = 0$.

The Chromatic Function Is a Polynomial!

Fundamental Recurrence for the Chromatic Function

$$\chi_G(k) = \begin{cases} k^{n(G)} & \text{if } m(G) = 0 \\ 0 & \text{if } G \text{ has loops} \\ \chi_{G \setminus e}(k) - \chi_{G/e}(k) & \text{if } e \text{ is any non-loop edge} \end{cases} .$$

The Chromatic Function Is a Polynomial!

Fundamental Recurrence for the Chromatic Function

$$\chi_G(k) = \begin{cases} k^{n(G)} & \text{if } m(G) = 0 \\ 0 & \text{if } G \text{ has loops} \\ \chi_{G \setminus e}(k) - \chi_{G/e}(k) & \text{if } e \text{ is any non-loop edge} \end{cases} .$$

Corollary

Given any loopless graph G , the function $\chi_G(k)$ agrees with a polynomial in $\mathbb{Z}[k]$ with no constant term and leading term $k^{n(G)}$.

The Chromatic Function Is a Polynomial!

Fundamental Recurrence for the Chromatic Function

$$\chi_G(k) = \begin{cases} k^{n(G)} & \text{if } m(G) = 0 \\ 0 & \text{if } G \text{ has loops} \\ \chi_{G \setminus e}(k) - \chi_{G/e}(k) & \text{if } e \text{ is any non-loop edge} \end{cases} .$$

Corollary

Given any loopless graph G , the function $\chi_G(k)$ agrees with a polynomial in $\mathbb{Z}[k]$ with no constant term and leading term $k^{n(G)}$.

In fact, $\chi_G(k)$ is called the **chromatic polynomial** of G .

A Pair of Useful Properties

If $G_1, \dots, G_\kappa$ are the connected components of G , then

$$\chi_G(k) = \prod_{i=1}^{\kappa} \chi_{G_i}(k).$$

A Pair of Useful Properties

If $G_1, \dots, G_{\kappa}$ are the connected components of G , then

$$\chi_G(k) = \prod_{i=1}^{\kappa} \chi_{G_i}(k).$$

If G is loopless, then $\chi_G(k) = \chi_{G'}(k)$, where G' is the *simplification* of G , the graph obtained by deleting all parallel edges of G .

Evaluation at $k = -1$

Since $\chi_G(\cdot)$ agrees with a polynomial, we can abstract away from the definition in terms of colorings and evaluate it at any number.

What happens if we evaluate $\chi_G(k)$ at $k = -1$?

Evaluation at $k = -1$

Since $\chi_G(\cdot)$ agrees with a polynomial, we can abstract away from the definition in terms of colorings and evaluate it at any number.

What happens if we evaluate $\chi_G(k)$ at $k = -1$?

Exercise

For every graph G ,

$$(-1)^{n(G)} \chi_G(-1) \geq 0.$$

Evaluation at $k = -1$

Since $\chi_G(\cdot)$ agrees with a polynomial, we can abstract away from the definition in terms of colorings and evaluate it at any number.

What happens if we evaluate $\chi_G(k)$ at $k = -1$?

Exercise

For every graph G ,

$$(-1)^{n(G)} \chi_G(-1) \geq 0.$$

Does it make sense to say that this function is a counting problem?

Evaluation at $k = -1$

Since $\chi_G(\cdot)$ agrees with a polynomial, we can abstract away from the definition in terms of colorings and evaluate it at any number.

What happens if we evaluate $\chi_G(k)$ at $k = -1$?

Exercise

For every graph G ,

$$(-1)^{n(G)} \chi_G(-1) \geq 0.$$

Does it make sense to say that this function is a counting problem?

The function $G \mapsto (-1)^{n(G)} \chi_G(-1)$ belongs to $\#P$.

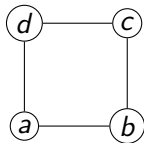
Why? How could we prove something like this?

Acyclic Orientations

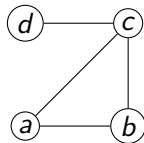
An **acyclic orientation** of a graph G is an assignment of a direction to each edge so that no directed cycle is formed. We denote the number of acyclic orientations of G by $\#AO(G)$.

Acyclic Orientations

An **acyclic orientation** of a graph G is an assignment of a direction to each edge so that no directed cycle is formed. We denote the number of acyclic orientations of G by $\#AO(G)$.

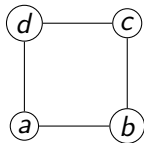


$$\#AO(G) = 2^4 - 2 = 14$$

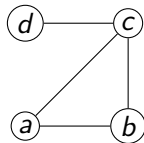


Acyclic Orientations

An **acyclic orientation** of a graph G is an assignment of a direction to each edge so that no directed cycle is formed. We denote the number of acyclic orientations of G by $\#AO(G)$.



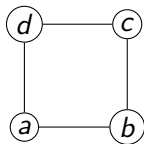
$$\#AO(G) = 2^4 - 2 = 14$$



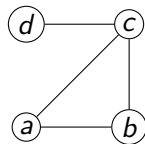
$$\#AO(G) = 2^4 - 2 \cdot 2 = 12$$

Acyclic Orientations

An **acyclic orientation** of a graph G is an assignment of a direction to each edge so that no directed cycle is formed. We denote the number of acyclic orientations of G by $\#AO(G)$.



$$\#AO(G) = 2^4 - 2 = 14$$



$$\#AO(G) = 2^4 - 2 \cdot 2 = 12$$

Exercise

$\#AO(G) > 0$ if and only if G has no loops.

Deletion-Contraction for Acyclic Orientations

Deletion-Contraction Recurrence for Acyclic Orientations

$$\#AO(G) = \begin{cases} 1 & \text{if } m(G) = 0 \\ 0 & \text{if } G \text{ has loops} \\ \#AO(G \setminus e) + \#AO(G/e) & \text{if } e \in E \text{ is not a loop} \end{cases} .$$

Deletion-Contraction for Acyclic Orientations

Deletion-Contraction Recurrence for Acyclic Orientations

$$\#AO(G) = \begin{cases} 1 & \text{if } m(G) = 0 \\ 0 & \text{if } G \text{ has loops} \\ \#AO(G \setminus e) + \#AO(G/e) & \text{if } e \in E \text{ is not a loop} \end{cases}.$$

Proof (idea).

Suppose that e connects the vertices u and v . Define two sets:

- A_1 : acyclic orientations of $G \setminus e$ for which there is a directed path from u to v or from v to u (but not both).
- A_2 : acyclic orientations of $G \setminus e$ for which u and v are mutually unreachable.

Deletion-Contraction for Acyclic Orientations

Deletion-Contraction Recurrence for Acyclic Orientations

$$\#AO(G) = \begin{cases} 1 & \text{if } m(G) = 0 \\ 0 & \text{if } G \text{ has loops} \\ \#AO(G \setminus e) + \#AO(G/e) & \text{if } e \in E \text{ is not a loop} \end{cases}.$$

Proof (idea).

Suppose that e connects the vertices u and v . Define two sets:

- A_1 : acyclic orientations of $G \setminus e$ for which there is a directed path from u to v or from v to u (but not both).
- A_2 : acyclic orientations of $G \setminus e$ for which u and v are mutually unreachable.

We have $\#AO(G) = |A_1| + 2|A_2| = \#AO(G \setminus e) + |A_2|$.

Deletion-Contraction for Acyclic Orientations

Deletion-Contraction Recurrence for Acyclic Orientations

$$\#AO(G) = \begin{cases} 1 & \text{if } m(G) = 0 \\ 0 & \text{if } G \text{ has loops} \\ \#AO(G \setminus e) + \#AO(G/e) & \text{if } e \in E \text{ is not a loop} \end{cases}.$$

Proof (idea).

Suppose that e connects the vertices u and v . Define two sets:

- A_1 : acyclic orientations of $G \setminus e$ for which there is a directed path from u to v or from v to u (but not both).
- A_2 : acyclic orientations of $G \setminus e$ for which u and v are mutually unreachable.

We have $\#AO(G) = |A_1| + 2|A_2| = \#AO(G \setminus e) + |A_2|$.

Finally, we will see that there is a bijection between A_2 and the acyclic orientations of G/e . □

A bijection between A_2 and the acyclic orientations of G/e

Let w be the vertex that results from contracting the edge e in G .

Given an orientation $\sigma \in A_2$, define $f(\sigma)$ as the orientation of G/e induced by σ after contracting e .

A bijection between A_2 and the acyclic orientations of G/e

Let w be the vertex that results from contracting the edge e in G .

Given an orientation $\sigma \in A_2$, define $f(\sigma)$ as the orientation of G/e induced by σ after contracting e .

Let us see that $f(\sigma)$ is acyclic. If it has a cycle, then this cycle contains w (since σ was an acyclic orientation of $G \setminus e$). If $w, v_1, \dots, v_\ell$ is a cycle in $f(\sigma)$, then, because σ is acyclic, necessarily $v_1, \dots, v_\ell$ is a directed path (in σ) from u to v or from v to u . But u and v are mutually unreachable in σ .

A bijection between A_2 and the acyclic orientations of G/e

Let w be the vertex that results from contracting the edge e in G .

Given an orientation $\sigma \in A_2$, define $f(\sigma)$ as the orientation of G/e induced by σ after contracting e .

Let us see that $f(\sigma)$ is acyclic. If it has a cycle, then this cycle contains w (since σ was an acyclic orientation of $G \setminus e$). If $w, v_1, \dots, v_\ell$ is a cycle in $f(\sigma)$, then, because σ is acyclic, necessarily $v_1, \dots, v_\ell$ is a directed path (in σ) from u to v or from v to u . But u and v are mutually unreachable in σ .

f is injective. What about surjectivity?

A bijection between A_2 and the acyclic orientations of G/e

Let w be the vertex that results from contracting the edge e in G .

Given an orientation $\sigma \in A_2$, define $f(\sigma)$ as the orientation of G/e induced by σ after contracting e .

Let us see that $f(\sigma)$ is acyclic. If it has a cycle, then this cycle contains w (since σ was an acyclic orientation of $G \setminus e$). If $w, v_1, \dots, v_\ell$ is a cycle in $f(\sigma)$, then, because σ is acyclic, necessarily $v_1, \dots, v_\ell$ is a directed path (in σ) from u to v or from v to u . But u and v are mutually unreachable in σ .

f is injective. What about surjectivity?

Let τ be an acyclic orientation of G/e . We construct an orientation σ of $G \setminus e$ as follows: each edge between $x \in V$ and u or v is oriented according to how the edges between x and w are oriented in τ , and for the other edges we copy the orientation. In σ , the vertices u and v are mutually unreachable; otherwise τ would not be acyclic. Thus $\sigma \in A_2$ and $f(\sigma) = \tau$.

The Bridge Between $\#AO$ and the Chromatic Polynomial

Theorem (R. Stanley, 1973)

$$(-1)^{n(G)} \chi_G(-1) = \#AO(G)$$

The Bridge Between $\#AO$ and the Chromatic Polynomial

Theorem (R. Stanley, 1973)

$$(-1)^{n(G)} \chi_G(-1) = \#AO(G)$$

Induction on $m(G)$. If $m(G) = 0$, then $\chi_G(k) = k^{n(G)}$ and

$$(-1)^{n(G)} \chi_G(-1) = (-1)^{2n(G)} = 1 = \#AO(G).$$

The Bridge Between $\#AO$ and the Chromatic Polynomial

Theorem (R. Stanley, 1973)

$$(-1)^{n(G)} \chi_G(-1) = \#AO(G)$$

Induction on $m(G)$. If $m(G) = 0$, then $\chi_G(k) = k^{n(G)}$ and

$$(-1)^{n(G)} \chi_G(-1) = (-1)^{2n(G)} = 1 = \#AO(G).$$

Now suppose that $m(G) \geq 1$. If G has loops, then both sides are equal to 0.

The Bridge Between $\#AO$ and the Chromatic Polynomial

Theorem (R. Stanley, 1973)

$$(-1)^{n(G)} \chi_G(-1) = \#AO(G)$$

Induction on $m(G)$. If $m(G) = 0$, then $\chi_G(k) = k^{n(G)}$ and

$$(-1)^{n(G)} \chi_G(-1) = (-1)^{2n(G)} = 1 = \#AO(G).$$

Now suppose that $m(G) \geq 1$. If G has loops, then both sides are equal to 0. Otherwise, let $e \in E$ be a non-loop edge. Then

$$\begin{aligned} (-1)^{n(G)} \chi_G(-1) &= (-1)^{n(G)} (\chi_{G \setminus e}(-1) - \chi_{G/e}(-1)) \\ &= (-1)^{n(G)} \chi_{G \setminus e}(-1) + (-1)^{n(G)-1} \chi_{G/e}(-1) \\ &= \#AO(G \setminus e) + \#AO(G/e) = \#AO(G) \end{aligned}$$

#P and Combinatorial Interpretations

Corollary (now we can!)

The function $G \mapsto (-1)^{n(G)} \chi_G(-1)$ belongs to #P.

#P and Combinatorial Interpretations

Corollary (now we can!)

The function $G \mapsto (-1)^{n(G)} \chi_G(-1)$ belongs to #P.

We can compute the function $G \mapsto (-1)^{n(G)} \chi_G(-1)$ by **counting objects**. The value of this function has a **combinatorial interpretation**.

#P and Combinatorial Interpretations

Corollary (now we can!)

The function $G \mapsto (-1)^{n(G)} \chi_G(-1)$ belongs to #P.

We can compute the function $G \mapsto (-1)^{n(G)} \chi_G(-1)$ by **counting objects**. The value of this function has a **combinatorial interpretation**.

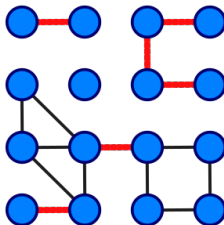
What have we just done?

- We considered two problems of different nature that satisfy deletion-contraction recurrences.
- We added factors to intertwine the recurrences.
- We proved an equality connecting the two functions.

This gives us a bridge for transferring complexity results between the two problems.

Loops, Coloops, and Ordinary Edges

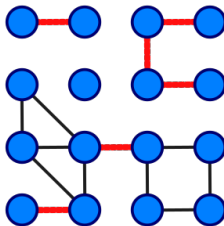
$e \in E$ is a **coloop** (or *bridge*, or *isthmus*) of G if $\kappa(G \setminus e) > \kappa(G)$.



User: Booyabazooka, Wikimedia Commons, CC BY-SA 3.0

Loops, Coloops, and Ordinary Edges

$e \in E$ is a **coloop** (or *bridge*, or *isthmus*) of G if $\kappa(G \setminus e) > \kappa(G)$.



User: Booyabazooka, Wikimedia Commons, CC BY-SA 3.0

We say that $e \in E$ is **ordinary** if it is neither a loop nor a coloop.

$c(G)$ denotes the number of coloops of G , and $\ell(G)$ denotes the number of loops. Clearly, G has ordinary edges if and only if $c(G) + \ell(G) < m(G)$.

The Reliability Polynomial

Let $0 < p < 1$ be a real number. We define $R_G(p)$ as the probability that the number of connected components of G does not increase if each edge is deleted with probability p and retained with probability $1 - p$ (independently).

The Reliability Polynomial

Let $0 < p < 1$ be a real number. We define $R_G(p)$ as the probability that the number of connected components of G does not increase if each edge is deleted with probability p and retained with probability $1 - p$ (independently).

Deletion-Contraction Recurrence for the Reliability Function

$$R_G(p) = \begin{cases} p \cdot R_{G \setminus e}(p) + (1 - p) \cdot R_{G/e}(p) & \text{if } e \text{ is an ordinary edge} \\ (1 - p)^{c(G)} & \text{if } c(G) + \ell(G) = m(G) \end{cases}.$$

The Reliability Polynomial

Let $0 < p < 1$ be a real number. We define $R_G(p)$ as the probability that the number of connected components of G does not increase if each edge is deleted with probability p and retained with probability $1 - p$ (independently).

Deletion-Contraction Recurrence for the Reliability Function

$$R_G(p) = \begin{cases} p \cdot R_{G \setminus e}(p) + (1 - p) \cdot R_{G/e}(p) & \text{if } e \text{ is an ordinary edge} \\ (1 - p)^{c(G)} & \text{if } c(G) + \ell(G) = m(G) \end{cases}.$$

Corollary

$R_G(p)$ is a polynomial in p with integer coefficients.

$R_G(p)$ is known as the **reliability polynomial** of G .

Expanded Definition of the Tutte Polynomial

The **Tutte polynomial** of G is the following polynomial in $\mathbb{Z}[x, y]$:

$$T_G(x, y) = \sum_{A \subseteq E} (x - 1)^{\kappa_G(A) - \kappa_G(E)} (y - 1)^{\kappa_G(A) + |A| - n(G)},$$

where $\kappa_G(A)$ denotes the number of connected components of the graph (V, A) .

Expanded Definition of the Tutte Polynomial

The **Tutte polynomial** of G is the following polynomial in $\mathbb{Z}[x, y]$:

$$T_G(x, y) = \sum_{A \subseteq E} (x - 1)^{\kappa_G(A) - \kappa_G(E)} (y - 1)^{\kappa_G(A) + |A| - n(G)},$$

where $\kappa_G(A)$ denotes the number of connected components of the graph (V, A) .

The Tutte polynomial of the loop graph $\mathcal{L}$

$$T_{\mathcal{L}}(x, y) = \underbrace{(x - 1)^{1-1} (y - 1)^{1+0-1}}_{A=\emptyset} + \underbrace{(x - 1)^{1-1} (y - 1)^{1+1-1}}_{A=E(\mathcal{L})} = y.$$

Expanded Definition of the Tutte Polynomial

The **Tutte polynomial** of G is the following polynomial in $\mathbb{Z}[x, y]$:

$$T_G(x, y) = \sum_{A \subseteq E} (x-1)^{\kappa_G(A) - \kappa_G(E)} (y-1)^{\kappa_G(A) + |A| - n(G)},$$

where $\kappa_G(A)$ denotes the number of connected components of the graph (V, A) .

The Tutte polynomial of the loop graph $\mathcal{L}$

$$T_{\mathcal{L}}(x, y) = \underbrace{(x-1)^{1-1} (y-1)^{1+0-1}}_{A=\emptyset} + \underbrace{(x-1)^{1-1} (y-1)^{1+1-1}}_{A=E(\mathcal{L})} = y.$$

The Tutte polynomial of the coloop graph $\mathcal{I}$

$$T_{\mathcal{I}}(x, y) = \underbrace{(x-1)^{2-1} (y-1)^{2+0-2}}_{A=\emptyset} + \underbrace{(x-1)^{1-1} (y-1)^{1+1-2}}_{A=E(\mathcal{I})} = x.$$

The Fundamental Recurrence for the Tutte Polynomial

Loop Factorization

If $e \in E$ is a loop, then

$$T_G(x, y) = y \cdot T_{G \setminus e}(x, y).$$

Coloop Factorization

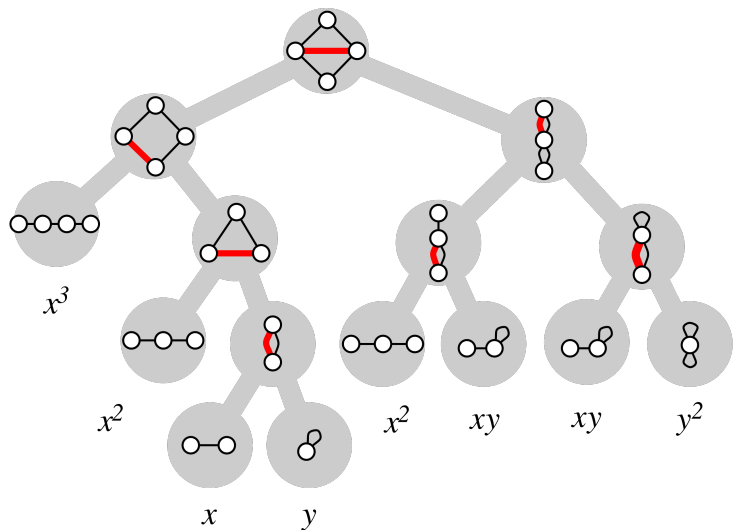
If $e \in E$ is a coloop, then

$$T_G(x, y) = x \cdot T_{G/e}(x, y).$$

Deletion-Contraction

$$T_G(x, y) = \begin{cases} T_{G \setminus e}(x, y) + T_{G/e}(x, y) & \text{if } e \text{ is an ordinary edge} \\ x^{c(G)} y^{\ell(G)} & \text{if } c(G) + \ell(G) = m(G) \end{cases}.$$

An Example of the Tutte Polynomial Recurrence



User: Thore Husfeldt, Wikimedia Commons, CC BY-SA 3.0

A Pair of Interesting Properties

Factorization

If $G = G_1 \cup G_2$, where G_1 and G_2 share at most one vertex, then

$$T_G(x, y) = T_{G_1}(x, y) \cdot T_{G_2}(x, y).$$

A Pair of Interesting Properties

Factorization

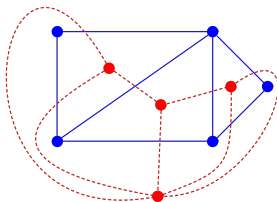
If $G = G_1 \cup G_2$, where G_1 and G_2 share at most one vertex, then

$$T_G(x, y) = T_{G_1}(x, y) \cdot T_{G_2}(x, y).$$

Planar Graph Duality

If G is planar and connected, and G^* is its dual graph, then

$$T_{G^*}(x, y) = T_G(y, x).$$



User: Self, Wikimedia Commons, CC0

Everything Is in the Tutte Polynomial

Let $\mathcal{G}$ be the set of isomorphism classes of graphs.

Universal Property of the Tutte Polynomial

Let $F: \mathcal{G} \rightarrow \mathbb{C}[\alpha, \beta, \gamma, x, y]$ be a function satisfying the following:

- (i) $F(G) = \gamma^{n(G)}$ if G has no edges.
- (ii) $F(G) = x \cdot F(G/e)$ if e is a coloop.
- (iii) $F(G) = y \cdot F(G \setminus e)$ if e is a loop.
- (iv) $F(G) = \alpha \cdot F(G/e) + \beta \cdot F(G \setminus e)$ if e is an ordinary edge.

If α and β are nonzero, then the following equality holds:

$$F(G) = \gamma^{\kappa(G)} \alpha^{n(G) - \kappa(G)} \beta^{\kappa(G) + m(G) - n(G)} \cdot T_G \left(\frac{x}{\alpha}, \frac{y}{\beta} \right).$$

Some Evaluations of the Tutte Polynomial

The Chromatic Polynomial

$$\chi_G(k) = (-1)^{n(G)-\kappa(G)} k^{\kappa(G)} \cdot T_G(1-k, 0).$$

Some Evaluations of the Tutte Polynomial

The Chromatic Polynomial

$$\chi_G(k) = (-1)^{n(G)-\kappa(G)} k^{\kappa(G)} \cdot T_G(1-k, 0).$$

In particular, taking $k = 3$, the number of 3-colorings of a connected graph G is $|3 \cdot T_G(-2, 0)|$.

The Reliability Polynomial

$$R_G(p) = (1-p)^{n(G)-\kappa(G)} p^{\kappa(G)+m(G)-n(G)} \cdot T_G\left(1, \frac{1}{p}\right).$$

Some Evaluations of the Tutte Polynomial

The Chromatic Polynomial

$$\chi_G(k) = (-1)^{n(G)-\kappa(G)} k^{\kappa(G)} \cdot T_G(1-k, 0).$$

In particular, taking $k = 3$, the number of 3-colorings of a connected graph G is $|3 \cdot T_G(-2, 0)|$.

The Reliability Polynomial

$$R_G(p) = (1-p)^{n(G)-\kappa(G)} p^{\kappa(G)+m(G)-n(G)} \cdot T_G\left(1, \frac{1}{p}\right).$$

Examples of Point Evaluations

- $T_G(2, 1)$ is the number of acyclic edge subsets of G .
- If G is connected, $T_G(1, 1)$ is the number of spanning trees of G .
- $T_G(0, 2)$ is the number of totally cyclic orientations of G .

The Complexity of Computing a Specific Coefficient

The following problem is $\#P$ -complete:

Input: a graph G and two nonnegative integers i, j given in binary.

Output: the coefficient of the term $x^i y^j$ in $T_G(x, y)$.

The Complexity of Computing a Specific Coefficient

The following problem is $\#P$ -complete:

Input: a graph G and two nonnegative integers i, j given in binary.

Output: the coefficient of the term $x^i y^j$ in $T_G(x, y)$.

Proof.

It is clear from the deletion-contraction recurrence of $T_G(x, y)$ that the function has nonnegative outputs. If $i + j > m(G)$, the coefficient is 0. Otherwise, $i, j \in O(m(G))$.

The Complexity of Computing a Specific Coefficient

The following problem is $\#P$ -complete:

Input: a graph G and two nonnegative integers i, j given in binary.

Output: the coefficient of the term $x^i y^j$ in $T_G(x, y)$.

Proof.

It is clear from the deletion-contraction recurrence of $T_G(x, y)$ that the function has nonnegative outputs. If $i + j > m(G)$, the coefficient is 0. Otherwise, $i, j \in O(m(G))$.

For membership in $\#P$, we fix an order on the edges and apply deletion-contraction always to the first edge that remains ordinary. We can use nondeterminism to guess the branch of the recurrence (choosing between the deletion and contraction branches) and then verify whether we end in a graph with i coloops, j loops, and no ordinary edges, in which case we accept.

The Complexity of Computing a Specific Coefficient

The following problem is $\#P$ -complete:

Input: a graph G and two nonnegative integers i, j given in binary.

Output: the coefficient of the term $x^i y^j$ in $T_G(x, y)$.

Proof.

It is clear from the deletion-contraction recurrence of $T_G(x, y)$ that the function has nonnegative outputs. If $i + j > m(G)$, the coefficient is 0. Otherwise, $i, j \in O(m(G))$.

For membership in $\#P$, we fix an order on the edges and apply deletion-contraction always to the first edge that remains ordinary. We can use nondeterminism to guess the branch of the recurrence (choosing between the deletion and contraction branches) and then verify whether we end in a graph with i coloops, j loops, and no ordinary edges, in which case we accept.

Next we will see that $G \mapsto T_G(x, y)$ is $\#P$ -hard, from which it follows that this problem is also $\#P$ -hard. □

#P-Equivalence of the Tutte Polynomial

The function $G \mapsto T_G(x, y)$ is #P-equivalent.

#P-Equivalence of the Tutte Polynomial

The function $G \mapsto T_G(x, y)$ is #P-equivalent.

Proof.

For #P-hardness, it is enough to recall that we can recover the number of 3-colorings of G from $T_G(x, y)$.

#P-Equivalence of the Tutte Polynomial

The function $G \mapsto T_G(x, y)$ is #P-equivalent.

Proof.

For #P-hardness, it is enough to recall that we can recover the number of 3-colorings of G from $T_G(x, y)$.

For #P-easiness, we can make $O(m(G)^2)$ calls to the problem of computing coefficients of $T_G(x, y)$, and thereby recover the whole polynomial. □

The Dichotomy Theorem for the Tutte Polynomial

Theorem (Jaeger, Vertigan, and Welsh, 1990)

Let $x_0, y_0 \in \mathbb{Q}$. The function $G \mapsto T_G(x_0, y_0)$ is computable in polynomial time if (x_0, y_0) lies in the set

$$\left\{ (x, y) \in \mathbb{Q}^2 \mid xy = x + y \right\} \cup \left\{ (1, 1), (-1, -1), (0, -1), (-1, 0) \right\},$$

and is #P-hard otherwise.

The Dichotomy Theorem for the Tutte Polynomial

Theorem (Jaeger, Vertigan, and Welsh, 1990)

Let $x_0, y_0 \in \mathbb{Q}$. The function $G \mapsto T_G(x_0, y_0)$ is computable in polynomial time if (x_0, y_0) lies in the set

$$\left\{ (x, y) \in \mathbb{Q}^2 \mid xy = x + y \right\} \cup \left\{ (1, 1), (-1, -1), (0, -1), (-1, 0) \right\},$$

and is #P-hard otherwise.

Theorem (Vertigan and Welsh, 1992)

Outside the easy cases of the previous theorem, when G is restricted to be a planar bipartite graph, then the function $G \mapsto T_G(x_0, y_0)$ is computable in polynomial time if and only if $x_0 y_0 = x_0 + y_0 + 1$; otherwise, it remains #P-hard.

References I



R. P. Stanley,
Acyclic orientations of graphs,
Discrete Mathematics, 5(2):171–178, 1973.



F. Jaeger, D. L. Vertigan, and D. J. A. Welsh,
On the computational complexity of the Jones and Tutte polynomials,
Mathematical Proceedings of the Cambridge Philosophical Society, 108(1):35–53,
1990.



D. L. Vertigan and D. J. A. Welsh,
The computational complexity of the Tutte plane: the bipartite case,
Combinatorics, Probability and Computing, 1(2):181–187, 1992.

References II



J. A. Ellis-Monaghan and C. Merino,
Graph Polynomials and Their Applications I: The Tutte Polynomial,
in *Structural Analysis of Complex Networks*, Birkhäuser Boston, pp. 219–255, 2011.



A. J. Goodall,
The Tutte polynomial and related polynomials,
Lecture notes, Charles University, 2013/2014.